



STUDENT

0021-KBC

TENTAMEN

240112 DV2606 2110
Salstentamen (DI)

Kurskod	--
Bedömningsform	--
Starttid	12.01.2024 09:00
Sluttid	12.01.2024 14:00
Bedömningsfrist	--
PDF skapad	21.05.2024 13:18
Skapad av	Zandra Johansson

1 There are, in general, two approaches to communicate between the processors in a MIMD parallel computer: *shared-memory* and *message-passing*.

- Describe the main characteristics of each of the two approaches.
- Further, present some advantage and disadvantage for each of them.
- Finally, give some example of parallel applications (with motivation) where shared-memory and message-passing would be preferable over the other, respectively.

Shared memory.

1. Can be either UMA or NUMA (Uniform or non uniform memory access). All processors are able to access (read/write to shared memory). In UMA the "distance" from each processor to the shared memory is the same and in NUMA the distance varies.
2. An advantage to shared memory machines is that there is less overhead associated with passing data around. A downside is that there is a need for atomic operations due to the problem of concurrent read/writes and also cache coherence problems.
3. It is beneficial for tasks that require high data granularity and reduced the need to pass data around. Examples are matrix multiplication and inplace algorithms like quicksort.

Message passing

1. Processors have their own memory (and possibly other devices) and share a communication bus on which they exchange messages and data. No processor is allowed to write or read from another processors memory and relies on this information being distributed by the owning processor.
2. An advantage is that this approach scales well since the cost of developing specialized hardware is too high. A disadvantage is that there is a high overhead associated with message passing and moving data around.
3. Message passing is suitable for computeclusters where very much compute needs to be controlled. Applications where there is little need for a high data granularity are suitable for message passing. One example could be separate databases that need some kind of processing and then sharing the (reduced) output.

Ord: 240

Besvarad.

- 2 When we parallelize a sequential application, e.g., by using threads or message-passing, so it can execute in parallel on N processors, we often find that the parallel program is not N times faster. Describe two important reasons for that.

Overhead

- There is some overhead associated with creating threads or message passing such as allocating space on the stack for the particular thread.

Read/write

- Having several threads work on the same data often creates a problem of concurrent read/writes or write/writes. In order to handle those conflicts synchronization primitives like mutex are used. That results in parts of the code not being able to run parallel and thus a sublinear speedup.

Ord: 71

Besvarad.

- 3 Sometimes we find that a system or application has a *superlinear speedup*. What does that mean?

Describe two situations when it may occur.

Less total work

- During a depth first tree search an implementation where different branches are handled by different threads often have to do less work than if the entire tree is handled by a single thread.

Better cache hit-ratio

- If the data that the application is handling can be divided among more processors and there is less data that needs to be moved to each cache it results in less cache collisions and therefore less time spent on data transfers.

Ord: 80

Besvarad.

4 When writing a parallel application, we can use mainly two approaches: *task (a.k.a. functional) parallelism* and *data parallelism*.

- Describe the main characteristics of each of the two approaches.
- Further, describe some advantage and disadvantage with each of them.
- Finally, give some examples of applications (with motivation) that are suitable for task parallelism and data parallelism, respectively.

Data parallelism

- Data parallelism is where the parallel work is done on a shared data structure. Akin to having multiple workers dig one big hole.
- One advantage is that it is suitable for SIMD and therefore scales well on modern gpus since they can share the instruction handling. This leads to data parallelism having high granularity. One disadvantage is that data parallelism is not suitable where tasks vary and there is many small but different data structures to work on. Furthermore the programmer needs to take care so that good memory access and work division is applied to avoid problems such as bad coalescing and thread divergence.
- Large matrix multiplication is a good example of an application where data parallelism is good because the data varies but the instructions are the same.

Task parallelism

- Task parallelism is when the parallel work is carried out on tasks that are independent of each other. Akin to having multiple workers dig multiple small holes.
- An advantage is that task parallelism is suitable for applications that handle varying problem sizes and loads. A disadvantage is that separating work on the basis of non overlapping tasks is often coarse grained and leads to lower granularity which can cause load balancing problems.
- A web server is an example where task parallelism is a suitable since each request is independent of the others. The amount of data that needs work is also unknown and that makes it difficult to divide the work on the basis of data.

Ord: 250

Besvarad.

5 Consider the following (pseudo) code segments with critical sections.

Initially, we assume initially that:

```
bool flag1 = false;  
bool flag2 = false;  
pthread_mutex_t mtx = unlocked;
```

Thread 1:

```
/// do something  
pthread_mutex_lock(&mtx)  
    while (!flag1);  
    flag2 = true;  
pthread_mutex_unlock(&mtx)  
/// do something more
```

Thread 2:

```
/// do something  
pthread_mutex_lock(&mtx)  
    while (!flag2);  
    flag1 = true;  
pthread_mutex_unlock(&mtx)  
/// do something more
```

Is this code correct or not? Clearly motivate why / why not!

The code will result in a spinlock where one thread is stuck in a while loop and the other waiting to enter a critical section.

1. As soon as one of the threads has acquired the lock the other must wait.
2. Since the flags are both false to begin with the while loops will be entered regardless of which thread enters the critical section first.
3. The thread that did not acquire the lock will not be able to enter the critical section and is left waiting.
4. The thread that entered the critical section will be stuck in the while loop indefinitely since while(!flag_x) will always be true.

Ord: 106

Besvarad.

6 Cache coherence is an important issue to handle in a shared-memory multiprocessor.

Describe, by giving an example, why ensuring cache coherence is important. (1p)

Two main approaches exist to maintain cache coherence: *write-invalidate* and *write-update*. Briefly describe each of them, e.g., by drawing a simple picture. (2p)

Describe some advantage and disadvantage with each of the approaches. (1p)

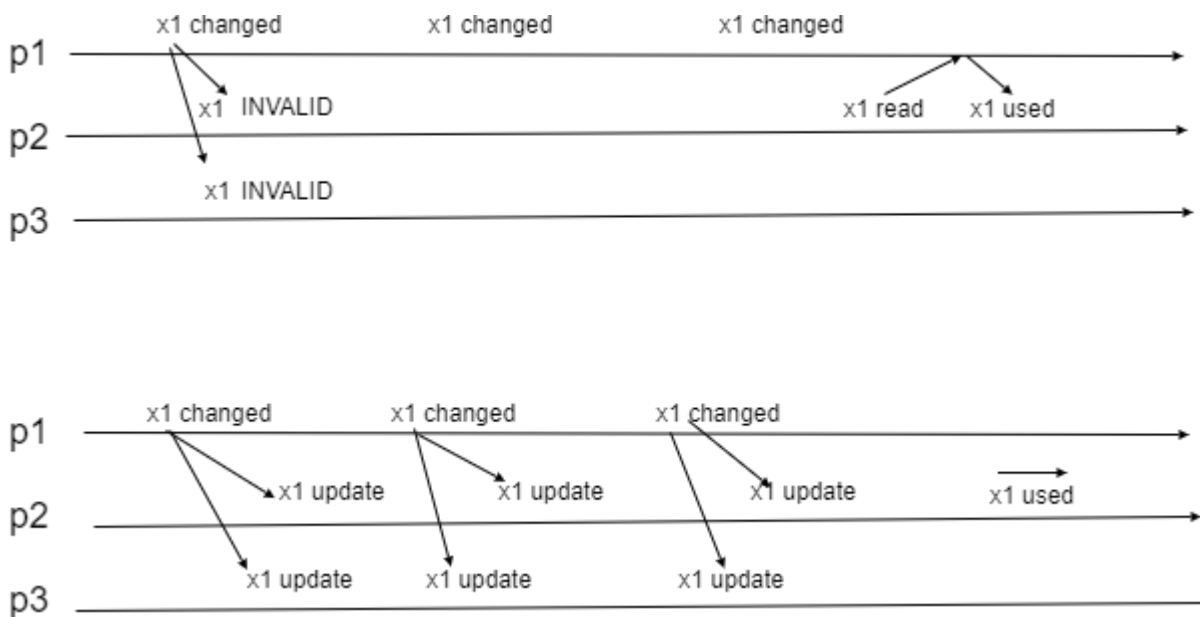
Cache coherence is important because if data is moved from shared memory to caches on different processors there exists a possibility that data that is supposed to represent the same value actually is different for different processors. Cache coherence is about ensuring that the computer acts a shared memory machine although the processors have different, and separate, caches.

Write-Invalidate

- This entails a processor that alters a shared variable to write to all other caches that their version of the variable is invalid and that they need to fetch it again before they use it. If a variable in cache is marked as invalid it is the responsibility of that processor to make sure to update its value before using it.
- Can introduce waits for the other processors that need to fetch altered data before proceeding but does not need to update every time the data is altered.

Write-Update

- This means that the processor who is altering a shared variable is responsible for updating the same variable in all other caches.
- Can create a lot of unnecessary overhead if there are many alterations of variables that are not used by others any more but does not create waits for the other processors.



An advantage to write-invalidate is that there is less write traffic when the number of processors is large and a downside is that it introduces waits for other processors.

An advantage to write-update is that the data is available right away when a processor needs it but a consequence is that the write traffic increases a lot which creates overhead.

Ord: 261

Besvarad.

7 Consider the following small parallel program, executed on two processors P_1 and P_2 , where the variables x_1 and x_2 are both initialized to 0:

Processor	P_1	P_2
Program	(1) $x_1 = 1$; (2) $\text{print}(x_2)$;	(3) $x_2 = 1$; (4) $\text{print}(x_1)$;

Which combination of values of x_1 and x_2 can be printed under sequential consistency, and why? (2p)

Which combination of values of x_1 and x_2 can be printed under weak ordering, and why? (2p)

Sequential consistency means that the order of execution of each program is strict but memory accesses among processors are arbitrary. Under sequential consistency the possible prints are therefore: 01,11
Weak ordering means that memory accesses does not need to be completed before the next instruction is performed. Then the possible outputs are: 00,01,10,11

Ord: 53

Besvarad.

8 The *PRAM (Parallel Random Access Machine)* model has often been criticized for being unrealistic. Why?
Why, and in which situations/contexts, does it still make sense to consider it?

PRAM does not take in to consideration that there is no global clock that synchronizes different processors and also does not take cache usage in to account.
It still makes sense to use PRAM since it is a simple and intuitive help for programmers to reason about algorithm performance. Even though the view is simplistic it is a helpful guide in designing code.

Ord: 63

Besvarad.

9

Thread coarsening is a technique that can improve performance in several situations. Thread coarsening can for instance help us to reduce the amount of shared memory needed and help us to reduce the total number of computations in the program. Explain what thread coarsening is. Give an example where thread coarsening helps us to reduce the amount of shared memory needed. Give an example where thread coarsening helps us to reduce the total number of computations in the program. Also, explain some possible drawbacks with thread coarsening.

What is thread coarsening

- Thread coarsening means that each thread does more work than otherwise. For instance a thread might operate on several elements instead of just one.

Example of reducing total number of computations

- One example of how thread coarsening lessens the amount of computations is summing a vector. If each thread operates on more than two elements in each warp there will be less warps but each warp will perform more work. This allows for less time being spent waiting for the other threads in the warp to finish in relation to the work performed.

Example of reducing the amount of memory accesses

- When summing an array thread coarsening can help to decrease the amount of writes to shared memory if each thread operates on more elements. Then the summing of a portion of the array can be performed in registers and only written to shared memory once that chunk is summed.

Drawbacks

- The drawback of thread coarsening is that the full capacity of the gpu is not used. In the most extreme case all the work is done by a single thread at which point the program is sequential.
- Thread coarsening can sometimes introduce memory accesses that are not well coalesced.

Ord: 204

Besvarad.

- 10** Consider a program with M input data points and N output data points. Each input and output point has a position and a value. Based on the positions of the data points and the values of the input data points we calculate the values of the output data points. Each input point affects the value of each output point. In this case we first add the value 5 to each input data point. If an input point is close to an output point, the value of the output point is affected stronger. The small sequential program below implements the exact algorithm:

```
for (m = 0; m < M; m++) {
    input[m].value += 5; /* Add the constant 5 to the value of each input data point */
    for (n = 0; n < N; n++) {
        /* the function distance calculates the distance between two data points */
        output[n].value += input[m].value/distance(input[m],output[n])
    }
}
```

The normal way to transform the sequential program into a parallel CUDA program is to replace the outer loop (i.e., the m-loop) with parallelism. In that case, what performance problems will the resulting CUDA program suffer from? Explain why the CUDA program will have performance problems. Also, explain how the sequential program can be transformed (into a functionally equivalent program) so that the resulting CUDA program will not suffer from these performance problems.

Concurrent writes

- If a CUDA program is implemented there will be write/write conflicts. Each thread can be assigned elements in the input array exclusively and write the number 5 but all of them will need to write to all of the output elements which will cause write conflicts. That means that they will need atomic operations and this will introduce a lot of overhead.

Scatter to gather

- We need to change the pattern from scatter to gather. Each thread should gather information from the input array and have exclusive writes in a part of the output array. Each thread should loop over the entire input array and add the value plus 5 divided by the distance value. Then the threads need to synchronize and after that a 5 can be added to each of the elements in the input array.

```
for (n = 0; n < N; n++) {
    for (m = 0; m < M; m++) {
        /* the function distance calculates the distance between two data points */
        output[n].value += (input[m].value+5)/distance((input[m]+5),output[n])
    }
}
for (m = 0; m < M; m++) {
    input[m].value += 5; /* Add the constant 5 to the value of each input data point */
}
```

Ord: 203

Besvarad.

11

Occupancy is defined as the ratio of active warps on an SM to the maximum number of active warps supported by the SM. Why do we normally want to have high occupancy? Which hardware related parameters affect occupancy? Discuss and give examples of how we as programmers, by taking smart design decisions, can provide good trade-offs between the parameters that affect occupancy.

We want high occupancy because otherwise there is wasted capacity in the SM. This means that the gpu might be working at full capacity but not doing as much useful work as it could.

Some hardware related parameters that affect occupancy are

1. The number of threads in a warp
2. The maximum number of warps in a block
3. The maximum number of blocks in a SM
4. The amount of registers available
5. the amount of shared memory in the SM

As a programmer i might want to use tiling to lessen amount of memory accesses from global memory. Depending on the size of the data used there will only be data for a certain number of threads on that SM since the shared memory is limited. This can result in the number of warps that are active is lower than what we want and we can change the size of the tile in order to facilitate for better occupancy.

Another case might be that the need for registers is large and this leads to thread coarsening. The coarsening factor should be chosen so that there are full warps, ie that all the threads in the warp are working.

Ord: 196

Besvarad.

12 Thread Blocks and Warps

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

- ☐ Threads in the same Warp can execute on different Streaming Multiprocessors.
- ☐ Threads from different Thread Blocks can execute in the same Warp.
- ☒ A Thread Block can contain many Warps. 
- ☒ Threads in the same Warp cannot execute different instructions at the same time. 
- ☐ A Thread Block cannot be larger than a Warp.

Rätt. 2 av 2 poäng.

13

What is control divergence? Why is control divergence bad for performance? Give an example how we as programmers can avoid or minimize control divergence.

Control divergence is when threads in the same warp take different paths.

If there is conditionals in the kernel and all threads in the warp does not take the same branch there will be divergence.

Control divergence is bad since the threads share the instruction and must all perform the same instruction at the same time. If there is divergence some threads will be muted and then reactivated. This means that part of the warp is idle and waiting.

As programmers we should strive to make each warp take the same control path. An example is using padding to make an array of a fixed size. This way the last thread does not have to calculate its own part of the array but will handle a part of the same size as all the other threads and the need for a conditional that checks wheter or not the thread is the last thread disappears.

Ord: 154

Besvarad.

14

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

- ☒ A CPU normally has larger caches than a GPU 
- ☐ The processor cores on a CPU are normally organized in streaming multiprocessors.
- ☒ Context switching on a GPU is normally faster than context switching on a CPU. 
- ☐ The processor cores in a CPU normally executes in SIMD mode.
- ☐ A CPU normally has more processor cores than a GPU

Rätt. 2 av 2 poäng.

15 Privatization

Privatization is a technique that can be used for improving the performance of some CUDA programs, e.g., programs that perform parallel histogramming. Explain how privatization works and why privatization can improve performance.

Privatization is when a thread has part of a solution private and shares it with others after it is done with its part of the work.

In the context of parallel histogramming the elements shall be classified as belonging to a certain class and the counter for that class shall be increased. Reading all elements and classifying them is not problematic but having several threads updating the same counters leads to write conflicts. In order not to have atomic operations each thread is responsible for histogramming a part of the data and that small histogram is then moved to shared memory.

For instance, a large array in global memory can be split in to shared memory in several SMs and many threads can work on part of the data and save counters in registers. Then the results are written to global memory and concatenated. This minimizes the amount of atomic operations needed.

Ord: 152

Besvarad.